P2013R5
Freestanding Language: Optional `::operator new`

Published Proposal, 2022-12-07

This version:
https://wg21.link/P2013R5

Author:
Ben Craig (NI and Boost Foundation)

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/ben-craig/freestanding_proposal/blob/master/core/new_delete.bs

---

Abstract

In freestanding implementations, standardize existing practices and make the default allocating `::operator new`s optional.

Table of Contents

1. Revision History

1.1. R5

LWG wording feedback.

1.2. R4

Definitions of the allocation functions must now exist, but they now have implementation defined behavior on freestanding implementations (which encompasses UB).

Rebasing to N4878. Accounting for shifted numbering in [new.delete].

1.3. R3

Added EWG Aug 2020 telecon polling results.

In the design, an ODR-use without a definition no longer requires ill-formedness. The intent is still the same, in that it typically results in a linker error.

Updated grammar and ill-formedness of wording.

Found an additional note in expr.new/10 that needed adjustment.

Deferred feature test macro decision to a future revision of P2198.

1.4. R2

Making the status of "placement" new more obvious ("placement" new is still required).

Added words in the design to indicate that the declarations in `<new>` are not changing.

Mentioning that pointer safety remains unchanged.

1.5. R1

Added polling results from Prague EWGI and EWG.

Added discussion on feature test macro.

Added discussion on not fixing virtual destructors.

Declaring that `constexpr new` must continue to work.

Added wording.

Wording includes changes to library clauses, so added LEWG to the audience.

1.6. R0

R0 of this paper was extracted from [P1105R1].

The proposed solution is different than the one proposed for P1105R1, but the motivation is the same. The solution from P1105R1 is still listed as a design alternative.

2. What is changing

On freestanding systems without default heap storage, the replaceable allocation functions (i.e. allocating `::operator new`, including the `nothrow_t` and `align_val_t` overloads, single and array forms) will have implementation defined behavior that does not need to do any kind of allocation. The implementation could return `nullptr`, invoke undefined behavior, or whatever the implementer likes. If a freestanding implementation provides one replaceable allocation function that meets the hosted requirements, then all the replaceable allocation functions shall meet the requirements. Hopefully, compiler vendors will warn on heap usage by default in freestanding implementations, and allow the warnings to be disabled for the situations where the user provides their own `::operator new`.

As a consequence of the above, runtime coroutines on freestanding implementations that are relying on the global allocation functions will also have implementation defined behavior, so long as the vendor supplied implementation is being used.

No other core language features require `::operator new`. basic.stc.dynamic.allocation

`::operator delete` will be implementable as a no-op function on implementations that have do-nothing or undefined-behavior `::operator new` implementations.

§ 3. What is staying the same

The replaceable deallocating `::operator delete` functions are still required to be present. `virtual` destructors ODR-use their associated `operator delete`(basic.def.odr), so keeping the global `::operator delete` allows those `virtual` destructors to continue building. Alternatives to this choice are discussed in § 7 Design Alternatives.

Calling `::operator delete` on a non-null pointer that did not come from `::operator new` is still undefined behavior new.delete.single new.delete.array. Calling `delete` on an object or base that didn't come from `new` is still undefined behavior expr.delete. This is what makes a no-op `::operator delete` a valid strategy for freestanding vendor supplied `::operator new` implementations.

The replaceable allocation functions will still be implicitly declared at global scope in each translation unit basic.stc.dynamic.general. A definition of the replaceable allocation functions must still exist. Non-ODR-uses of the replaceable allocation functions are still permitted (e.g. inside of uninstantiated templates). The declarations of `::operator new` in the `<new>` header are still required to be present. Implementations of the replaceable allocation functions can be performed by linking in an extra translation-unit with the definitions of the functions.

"`constexpr new`" is still required to work, even when the vendor supplied replaceable allocation functions have not been replaced. The calls to `::operator new` are required to be omitted, so any undefined behavior in the `::operator new` implementation will also be skipped.

Non-allocating placement `::operator new` (colloquially "placement new") and `::operator delete` (colloquially "placement delete") are still required to be present in freestanding implementations.

Core language concepts of pointer safety remain unchanged. Note that the pointer safety library facilities util.dynamic.safety are not required to be present in freestanding implementations, and the author is not aware of any papers to make the pointer safety library facilities required in freestanding implementations.

Hosted implementations are unchanged. Users of freestanding implementations can still provide implementations of the replaceable allocation and deallocation functions. The behavior of `virtual` destructors is unchanged. The behavior of class specific `operator new` and `operator delete` overloads is unchanged. The requirements on user-provided `::operator new` and `::operator`

`delete` overloads remains the same, particularly those requirements involving error behaviors. Coroutines will behave the same so long as promise-specific allocators are used. The storage for exception objects will remain unspecified.

4. Why?

4.1. No allocations allowed

In space constrained and/or real-time environments, there is often no free store. These environments often cannot tolerate the space overhead for the free store, or the non-determinism from using the free store. In these environments, it is a desirable property for accidental global `new` usage to generate a diagnostic. Compilers are capable of generating a diagnostic when they ODR-use a function (e.g. `::operator new`).

FreeRTOS allows for both static and dynamic allocation of OS constructs [FreeRTOS_StaticVDynamic]. Static allocation in conjunction with `::operator new` diagnostics can help avoid overhead and eliminate accidental usage.

THREADX [THREADX] does not consider dynamic allocation a core service, and can be built without support for dynamic allocation in order to reduce application size. THREADX also distinguishes between byte allocation (general purpose) vs. block allocation (no-fragmentation elements of fixed size in a pool).

Also, by allowing a no-op `::operator delete` implementation, these space constrained applications can save code-size. No code needs to be present for `::operator delete` synchronization, free block coalescing, or free block searching.

4.2. No right way to allocate memory

In some target environments, there is no "right" way to allocate memory. In kernel and embedded domains, the implementer of the C++ toolchain doesn’t always know the "right" way to allocate memory on the target environment. This makes it difficult to provide an implementation for `::operator new`. The implementer cannot even rely on the presence of `malloc`, as it runs into the same fundamental problems.

As an example, in the Microsoft Windows kernel environment, there are two leading choices about where to get dynamic memory [MSPools]. Users can get memory from the non-paged pool, which is a safe, but scarce resource; or users can get memory from the paged pool, which is plentiful, but not accessible in many common kernel operations. Non-paged pool must be used any time the allocated

memory needs to be accessible from an interrupt or from a "high IRQL" context. The author has had experience with both paged pool and non-paged pool as defaults, with the predictable outcome of crashes with paged pool defaults and OOM with non-paged pool defaults. The implementer of the C++ toolchain is not in a good position to make this choice for the user.

In the Linux kernel environment, `kmalloc` [kmalloc] with the `GFP_KERNEL` flag should be used when allocating memory within the context of a process and outside of a lock, but the `GFP_ATOMIC` flag should be used when allocating memory outside the context of a process, such as inside of an interrupt. The implementers of the C++ runtime are in no position to know which is the correct flag to use by default. Using `GFP_KERNEL` when `GFP_ATOMIC` is needed will result in crashes from interrupt code and deadlocks. Using `GFP_ATOMIC` when `GFP_KERNEL` is appropriate will result in reduced system performance, spurious OOM errors, and premature exhaustion of emergency memory pools.

Freestanding implementations are intended to run without the benefit of an operating system (intro.compliance). However, the name of the function that supplies dynamic memory is usually an OS-specific detail. The C++ implementation should not (and may not) know the name of the function to request memory. The Windows kernel uses `ExAllocatePoolWithTag`. In the Linux kernel, `kmalloc` is the main function to use. In FreeBSD, a function named `malloc` is present, but it takes different arguments than the C standard library function of the same name. FreeRTOS uses `pvPortMalloc`, and THREADX uses `tx_byte_allocate`. Home-grown OSes will likely have other spellings for memory allocation routines.

Today's C++ implementations don't provide `::operator new` implementations for all possible targets. Doing so isn't a plausible goal, especially when the home-grown OSes are taken into account. This means that users are already forced into choosing between not having `::operator new` support and providing their own implementation. We should acknowledge and standardize this existing practice, especially since we already have the extension point mechanism in place.

§ 4.3. What about allocators?

The C++20 freestanding library does not include allocators. [P1642R1] proposes adding allocator machinery to freestanding, but doesn't add `std::allocator` itself. In addition, none of the allocating standard containers are in C++20's freestanding library or any current freestanding library proposal that the author is aware of. From a minimalist freestanding perspective, allocators aren't a solution.

Allocators are still useful in a less-than-minimal freestanding implementation. In environments with dynamic memory, custom allocators can be written and used with standard containers, assuming that the containers are present in the implementation. This could be done even if a global `::operator`

`new` is not present. The author has used `stlport::vector<int, PageLockedAllocator>` successfully in these environments.

`std::allocator` is implemented in terms of global `::operator new`. In practice, it would be easy for an implementation to have an implementation of `std::allocator` in a header / module, and have that header still compile just fine. If the user has provided a global `::operator new`, then `std::allocator` would have the same semantics as mandated for hosted implementations. If the global `::operator new` is vendor supplied, then uses of `std::allocator` would invoke implementation defined behavior, and hopefully cause a diagnostic.

Some facilities in the standard library (e.g. `make_unique`) are implemented in terms of `new`, and not an allocator interface. It is useful to make these facilities generate diagnostics when dynamic memory isn't available, and it is also useful to be able to control which memory pool is used by default.

§ 4.4. `virtual` destructors

A no-op `::operator delete` is still provided in order to satisfy `virtual` destructors. `virtual` destructors ODR-use their associated `operator delete`(basic.def.odr). This approach has the disadvantage that there is a small, one-time overhead for the first `virtual` destructor in a program, even if there are no usages of `new` or `delete`. The overhead is small though, and you only pay for the overhead if you use `virtual` destructors.

Ideally, if neither `new` nor `delete` is ever called, we wouldn't need an `operator delete`. This proposal still requires some `operator delete` to exist, though that `operator delete` can be a no-op.

§ 4.5. Why not fix `virtual` destructors, instead of keeping a no-op `operator delete`?

This paper attempts to standardize existing practice. There is not any existing practice for "fixed" `virtual destructors`. Note that this paper isn't changing any requirements on `operator delete` or `virtual` destructors. It will be no more difficult to fix it in the future than it would be today. A motivated author could attempt to fix the problem in a future paper.

§ 4.6. Feature test macro

P2198R1 discusses the feature test macro in more depth, and provides a recommendation.

In order to provide this macro, library implementations are going to require knowledge of the target environment. That knowledge may be via a list of known target platforms that are detected at build time, or by having the builder of the implementation supply that information in a configuration flag.

The most likely usage of a feature test macro for this feature is to conditionally define a custom `::operator new` iff the implementation did not provide one by default. This is dangerous territory, as it encourages libraries to provide the one-and-only `::operator new` definition. If two such libraries do this, then there is an ODR issue.

Another likely usage is to fall back to an implementation that does not use the heap at all.

```
#if defined(__cpp_lib_no_default_operator_new) && __cpp_lib_no_default_oper
  using my_container = fixed_capacity_vector;
#else
  using my_container = my_vector;
#endif
```

This is an imprecise check. Even though there is no default operator new, there may be a user provided operator new that works fine.

If this feature test macro were provided in the positive (`__cpp_lib_has_default_operator_new`), it wouldn't be useful for a very long time.

```
#if defined(__cpp_lib_has_default_operator_new) && __cpp_lib_has_default_op
  using my_container = my_vector;
#else
  // spuriously triggers for C++20 and earlier code
  using my_container = fixed_capacity_vector;
#endif
```

If this feature test macro were provided in the negative (`__cpp_lib_no_default_operator_new`), it would be the only feature test macro that wouldn't be required to be defined in a conforming implementation.

All these considerations are deserving of dedicated SG10 discussion with P2198, and should not hold up the progress of this paper.

§ 4.7. Likely misuses and abuses

Users are likely to provide overloads of `::operator new` that do not follow the requirements set forth in new.delete, particularly the requirements around throwing `bad_alloc`. Ignoring this requirement will still result in undefined behavior, just as it does in C++20. Some compilers optimize assuming that the throwing forms of `new` will never return a null pointer [throwing_new]. A likely outcome of the undefined behavior is unexpectedly eliding null checks in the program source. This problem already exists today, and this proposal makes it no worse.

§ 5. Experience

The proposed design has field experience in a micro-controller environment. GCC was used, and the language support library was intentionally omitted. A no-op `::operator delete` was provided by the users. The no-op `::operator delete` enabled a small amount of code sharing between a hosted environment and this micro-controller environment. Some shared code involved classes with `virtual` destructors.

§ 6. Polling history

§ 6.1. Aug 19, 2020, EWG, Telecon

After updating wording for “either/of”, mandates, and talking to SG10 about the feature test macro, P2013 is tentatively ready to be forwarded to CWG. Walter volunteers to check that this is done.

SF/F/N/A/SA

2/16/1/0/0

§ 6.2. Feb 14, 2020, EWG, Prague

We are interested in freestanding having an optional operator new, please come back with wording

SF/F/N/A/SA

8/10/7/0/0

6.3. Feb 12, 2020, EWGI, Prague

Given the time constraints of the committee, should we spend additional committee effort on P2013?

SF/F/N/A/SA

7/6/1/0/0

Is a Feature test macro a valuable addition to this paper?

SF/F/N/A/SA

0/5/5/2/0

Do we believe that P2013 is sufficiently developed to be seen by EWG?

SF/F/N/A/SA

5/8/0/0/0

6.4. Jan 8, 2020 SG14 Telecon

Forward P2013 as is with the minor editing quotes

SF/F/N/A/SA

9/10/0/0/0

approves to go to EWG

7. Design Alternatives

7.1. Alternative 0: Implementation defined allocating `::operator new` (Proposed above)

This option preserves much functionality, without using any novel techniques. The main disadvantage of this approach compared to existing techniques is that diagnosing the problem requires more work. See above for further explanation.

§ 7.2. Alternative 1: Optional throwing `::operator new`s, no-op default deallocation functions

Rather than making all the replaceable allocation functions have implementation defined behavior, we could make just the throwing `::operator new`s implementation defined (array and single form, with and without `align_val_t` parameters). The library would still be required to meet the hosted requirements for `nothrow_t` overloads.

The `nothrow_t` overloads are specified to forward to an appropriate throwing overload. That implementation would still be fine on a system without dynamic storage available. This alternative was not selected as it is more difficult to teach, and because the target audience would likely be astonished that the `nothrow_t` overload has a `try`/`catch` in it.

§ 7.3. Alternative 2: No deallocation functions

The presence of the replaceable deallocation functions is implementation defined. `virtual` destructors will be ill-formed unless the implementation provides the deallocation function, the user provides a global `::operator delete` function, or the user provides a class specific `operator delete` overload.

This alternative has the benefit of being zero overhead and very explicit, but it has troublesome consequences for implementations. There are several language support classes that have `virtual` destructors, and something would need to be decided for them. Notably, `type_info` and the `exception` hierarchy all have `virtual` destructors. The standard library implementers may be prohibited from providing `operator new` and `operator delete` overloads (conforming#member.functions). Alternatively, the facilities that require classes with `virtual` destructors could all be off-limits until `operator delete` was made available. This would eliminate many cases with exceptions, `dynamic_cast` on references, and `typeid`.

If we were to adopt this alternative, many users would provide a no-op `::operator delete` in their code, giving their code the same semantics and trade-offs as the proposed solution.

§ 7.3.1. Experience

This alternative has field experience. MSVC's /kernel [kernel_switch] flag omits definitions for `::operator new` and `::operator delete`. Users of Clang and GCC can choose to not link against the language support library, and therefore not have `::operator new` and `::operator delete` support, as well as many other language support features.

7.4. Alternative 3: No deallocation functions and new ODR-used rules for virtual destructors

The presence of the replaceable deallocation functions is implementation defined. Change `virtual` destructors so that they generate a partial vtable and don't ODR-use `::operator delete`. Make `new` expressions ODR-use `::operator delete` and complete the vtable.

7.4.1. How could this virtual destructor ODR-use change be implemented?

First, this is only a problem that needs to be solved on systems without a default heap. This means that typical user-mode desktop and server implementations would be unaffected.

Existing linkers already have the ability to take multiple identical virtual table implementations and pick one for use in the final binary. A potential implementation strategy is for compilers and linkers to support a new "weaker" linkage. When the default heap is disabled, the compiler would emit a vtable with a `nullptr` or pure virtual function in the virtual destructor slot. When `new` is called, a "stronger" linkage vtable would be emitted that has the deleting destructor in the virtual destructor slot. The linker would then select a vtable with the strongest linkage available. Today's linkage would be considered "stronger". Only partially filled vtables would have "weaker" linkage.

7.4.2. ABI impact

Mixing multiple object files into the same program should be fine, even if some of them have a default heap and some don't. All the regular / "strong" linkage vtables should be identical, and all the "weaker" linkage vtables should be identical. If anyone in the program calls any form of `new`, the deleting destructor will be present and in the right slot. If no-one calls `new` in the program, then no-one should be calling `delete`, and the empty vtable slot won't be a problem.

Shared libraries are trickier. Vtables aren't always emitted into every translation unit. Take shared library "leaf" that has a default heap. It depends upon shared library "root" that does not have a default heap. If a class with a virtual destructor is defined in "root", along with its "key function", then a call to `new` on the class in "leaf" will generate an object with a partial vtable. Calling `delete` on that object will cause UB (usually crashes).

Lack of a default heap should generally be considered a trait of the platform. Mixing this configuration shouldn't be a common occurrence.

7.4.3. Experience

This alternative is novel, and does not have implementation or usage experience.

7.5. Alternative 4: Missing `::operator new`, no-op default deallocation functions (Proposed in R3)

Rather than have the behavior of the replaceable global allocation functions be implementation defined on freestanding implementations, we could instead have their presence be optional. At runtime, this is easy enough, as we can lean on ODR to do the work (zero definitions is not one definition). This results in an IF-NDR program, but in practice, you either get a linker error, or you get the "right" behavior, because the calls to `::operator new` were heap-elided.

This approach is substantially more difficult with `constexpr new` though. Colloquially, users would expect that uses of `::operator new` at compile time would not require a runtime definition of `::operator new`. However, `constexpr` doesn't work like that. Implementations are permitted to emit a runtime definition of `constexpr` functions, as well as keep an internal representation for compile time evaluation, even if the function only happens to be used for constant evaluation. The runtime definition could still cause linker errors, even if nothing is calling the function.

It is possible to rework the ODR rules such that a runtime definition is only emitted if needed. That is a substantially larger change though, and there was resistance from CWG to make this large of a change to ODR at this time. This approach should be reconsidered for adoption if more OS-dependent facilities become available at compile in the future (e.g. `constexpr` iostreams). A lower effort approach is acceptable for now though, since `::operator new` and `::operator delete` are the only such facilities at present.

8. Wording

This is based on the December working draft, N4878.

8.1. new.delete

Modify new.delete

> 17.6.3 Storage allocation and deallocation [new.delete]
>
> 17.6.3.1 General [new.delete.general]
>
> 1 Except where otherwise specified, the provisions of 6.7.5.5 apply to the library versions of operator new and operator delete. If the value of an alignment argument passed to any of these functions is not a valid alignment value, the behavior is undefined.
>
> 2 On freestanding implementations, it is implementation-defined whether the default versions of the replaceable global allocation functions satisfy the required behaviors described in [new.delete.single] and [new.delete.array].
>
> [*Note*: A freestanding implementation's default versions of the replaceable global allocation functions can cause undefined behavior when invoked. During constant evaluation, the behaviors of those default versions are irrelevant, as those calls are omitted ([expr.new]).- *end note*]
>
> *Recommended practice:* If any of the default versions of the replaceable global allocation functions meet the requirements of a hosted implementation, they all should.

§ 9. Acknowledgments

Thank you to the many reviewers of this paper: Brandon Streiff, Irwan Djajadi, Joshua Cannon, Brad Keryan, Alfred Bratterud, Phil Hindman, Arthur O'Dwyer, Laurin-Luis Lehning, JF Bastien, Matthew Bentley, and Alisdair Meredith.

Thank you to Daveed Vandevoorde and Walter Brown for providing feedback on the wording.

§ References

§ Informative References

[FreeRTOS_StaticVDynamic]
FreeRTOS Documentation. *Static Vs Dynamic Memory Allocation*. URL: https://www.freertos.org/Static_Vs_Dynamic_Memory_Allocation.html

[KERNEL_SWITCH]
Microsoft Documentation. */kernel (Create Kernel Mode Binary)*. URL: https://docs.microsoft.com/en-us/cpp/build/reference/kernel-create-kernel-mode-binary

[KMALLOC]

kernel.org. *kmalloc*. URL: https://www.kernel.org/doc/htmldocs/kernel-api/API-kmalloc.html

[MSPools]

Microsoft Documentation. *POOL_TYPE enumeration*. URL: https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/ne-wdm-_pool_type

[P1105R1]

Ben Craig; Ben Saks. *Leaving no room for a lower-level language: A C++ Subset*. URL: https://wg21.link/P1105R1

[P1642R1]

Ben Craig. *Freestanding Library: Easy [utilities], [ranges], and [iterators]*. URL: https://wg21.link/P1642R1

[THREADX]

THREADX(R) RTOS - Royalty Free Real-Time Operating System. URL: https://rtos.com/solutions/threadx/real-time-operating-system/

[THROWING_NEW]

Microsoft Documentation. */Zc:throwingNew (Assume operator new throws)*. URL: https://docs.microsoft.com/en-us/cpp/build/reference/zc-throwingnew-assume-operator-new-throws?view=vs-2019